

JWT Bearer Auth & Refresh Rotation

Token generation (TokenService) · Claims · AddJwtBearer pipeline · User
Secrets · Refresh Token Rotation

TokenService JWT

```
var token = new JwtSecurityToken(  
    issuer: _config["Jwt:Issuer"],  
    audience: _config["Jwt:Audience"],  
    claims: claims, expires:  
    DateTime.UtcNow.AddMinutes(15));
```

JWT Bearer Authentication & Refresh Token Rotation

1. Anatomy of a JSON Web Token (JWT)

- A JWT is a compact, URL-safe means of representing claims between two parties.
- Header: Algorithm (HMAC-SHA256) & Token Type (JWT).
- Payload: Claims (sub, email, role, FirstName, exp, iat). *Base64Url encoded, NOT encrypted.*
- Signature: Hash of Header + Payload signed with the server's symmetric secret key.

2. Token Issuance & ASP.NET Core Middleware Pipeline

TokenService.cs / Program.cs

```
// TokenService.cs
```

```
public string GenerateAccessToken(TmsUser user, IList<string> roles) {
```

```
    var claims = new List<Claim> { new Claim(JwtRegisteredClaimNames.Sub, user.Id), new Claim(JwtRegisteredClaimNames.Email, user.Email!), new Claim("FirstName", user.FirstName) };
```

```
    foreach (var role in roles) claims.Add(new Claim(ClaimTypes.Role, role));
```

```
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
```

```
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
```

```
    var token = new JwtSecurityToken(issuer: _config["Jwt:Issuer"], audience: _config["Jwt:Audience"], claims: claims, expires: DateTime.UtcNow.AddMinutes(15), signingCredentials: creds);
```

```
    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

```
// Program.cs
```

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(options => { ... });
```

```
app.UseAuthentication(); // MUST be placed before app.UseAuthorization()!
app.UseAuthorization();
```

3. Secret Hygiene

- Never commit raw JWT secret keys to appsettings.json or source control.

CLI Secret Management

```
# Set secret locally in developer secret store
```

```
dotnet user-secrets set "Jwt:Key" "EnterpriseSuperSecretSigningKey32BytesLong!"
```

4. Refresh Token Rotation & Theft Detection

- Short-lived access tokens (15 min) limit window of vulnerability if intercepted. Long-lived refresh tokens allow seamless UI token renewals via POST /api/auth/refresh.

```
[HttpPost("refresh")]
public async Task<IActionResult> Refresh([FromBody] TokenRefreshRequest request) {
    var storedToken = await _context.RefreshTokens.FirstOrDefaultAsync(r => r.Token == request.RefreshToken);

    if (storedToken == null) return Unauthorized(new { message = "Invalid refresh token." });

    // Theft Detection: If an already-used refresh token is presented again!
    if (storedToken.IsUsed) {
        var compromisedTokens = await _context.RefreshTokens.Where(r => r.UserId == storedToken.UserId).ToListAsync();
        foreach (var t in compromisedTokens) t.IsRevoked = true;
        await _context.SaveChangesAsync();
        return Unauthorized(new { message = "Security alert: Refresh token reuse detected. All sessions revoked." });
    }

    if (storedToken.IsRevoked || storedToken.ExpiryDate < DateTime.UtcNow) return Unauthorized(...);

    storedToken.IsUsed = true; // Single-use consumption

    var user = await _userManager.FindByIdAsync(storedToken.UserId);

    var roles = await _userManager.GetRolesAsync(user!);

    var newAccessToken = _tokenService.GenerateAccessToken(user!, roles);

    var newRefreshToken = _tokenService.GenerateRefreshToken(user!.Id);
    _context.RefreshTokens.Add(newRefreshToken);
    await _context.SaveChangesAsync();
    return Ok(new { token = newAccessToken, refreshToken = newRefreshToken.Token });
}
```

5. Session 2 Traps & Common Mistakes

Mistake	Symptom	Enterprise Fix
Missing app.UseAuthentication()	Controller returns 401 even with valid token	Call app.UseAuthentication() before app.UseAuthorization()
Committed JWT Key	Security scanners flag repository	Use dotnet user-secrets in dev, Environment variables in prod
No Single-Use Rotation	Refresh token can be reused infinitely	Mark IsUsed = true and revoke family on reuse detection

Next: M11 Session 3 — Policy Authorization, Angular Guards & API Hardening

Custom AuthorizationHandlers, Angular roleGuard, rate limiting, and security headers.